

IntelliCar

HYBRID VOICE-CONTROLLED ROVER

ECE 655L Final Project | Yihang Zou, Yuchu Guo, Yifan Xu



The Voice Control Dilemma

Cloud-Only

Pros: Powerful understanding and context awareness.

Cons: High latency makes real-time navigation dangerous.
Dependent on internet stability.

Local-Only

Pros: Extremely fast execution (milliseconds).

Cons: Rigid command structure.
Fails to understand natural, conversational language.



To create an intelligent system
embedded in car that combines
“the low latency of local processing”
with “the natural language
understanding of Cloud AI”.

- Our Mission, from Yuchu Guo

System Architecture

1. Input & VAD

- USB Microphone captures audio streams
- Local algorithm (Vosk) clips valid speech segments dynamically.

2. Local STT

- **Vosk** handles keyword spotting and grammar checking.
- **Whisper** provides high-fidelity transcription.

3. Intelligent Dispatch

- Dual Vosk paths (Grammar & No-Grammar) run in parallel.
- Perfect matches → local execution: otherwise Whisper/LLM fallback.

4. Motion Execution

- Local driver executes JSON commands.
- Uses MPU6050 for closed-loop turning + overshoot prevention and drift correction.

Core Hardware Components

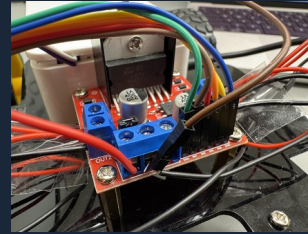
Compute

Raspberry Pi 4B
(8GB RAM) for
orchestration.



Drive

L298N Dual H-
Bridge Motor
Driver.



Sensor

MPU-6050
Gyroscope (for
precise
turning)



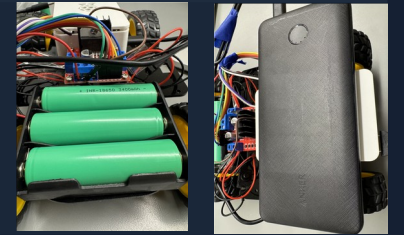
Input

USB Conference Microphone.



Power

5V/3A Bank (Pi) + 12V Battery
Pack (Motors).



Software Stack

Python

The core controller managing threading, VAD logic, and GPIO control signals.

Vosk & Whisper

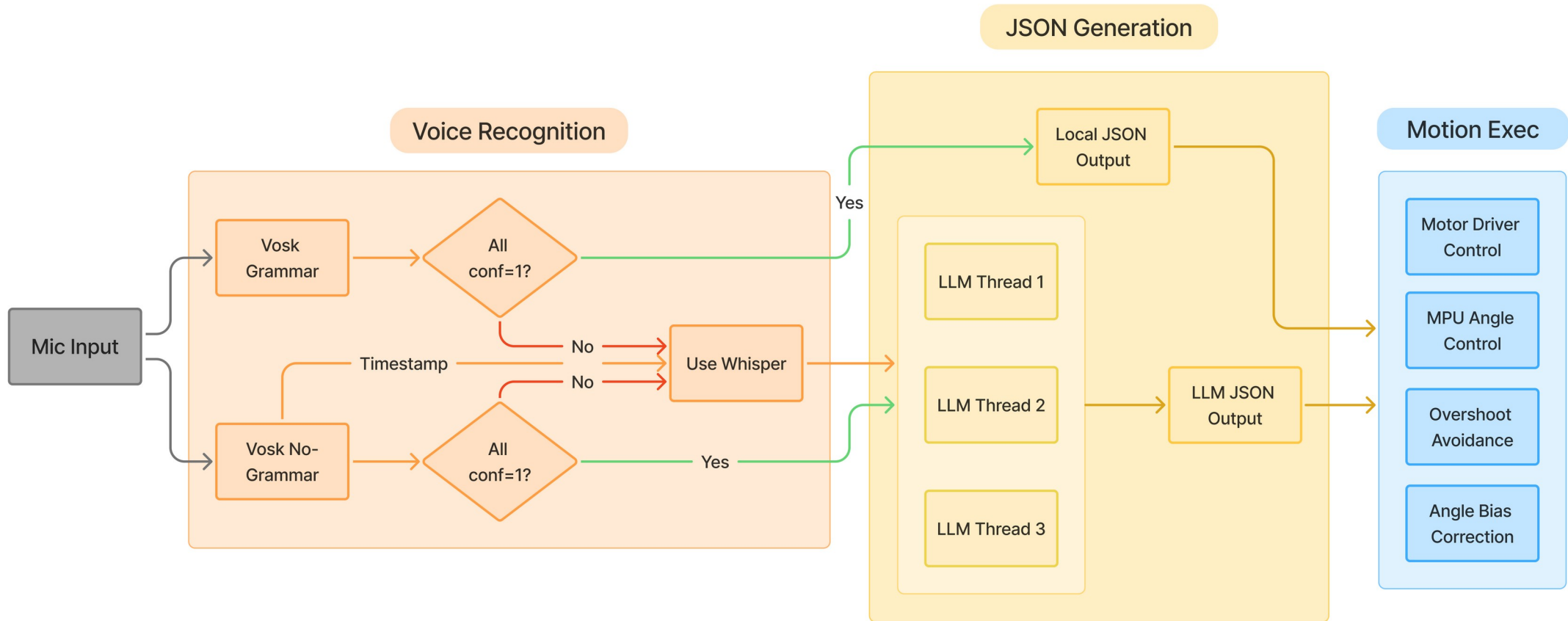
- Vosk: Zero-latency keyword spotting.
- Whisper: High-fidelity local transcription.

OpenAI API

Processes complex natural language and returns structured JSON commands.

Methodology

Overall Workflow



Adaptive Voice Recognition

Fast Path (Grammar Perfect)

Trigger:

Vosk Grammar Conf == 1

Logic:

Detects only structured keywords in grammar mode.

Low-latency recognition specialized for strict command words.

Use When:

Short, clear control phrases

Flow:

Vosk Grammar → Keywords

Direct Path (No-Grammar Perfect)

Trigger:

Vosk No-Grammar Conf == 1

Logic:

Free-form recognition suitable for more natural phrasing.

The transcript is used directly without Whisper enhancement.

Use When:

Clean, fluent sentences

Flow:

Vosk No-Grammar → Natural Language

Whisper Upgrade Path (Fallback)

Trigger:

Either Vosk not perfectly confident

Logic:

Vosk provides precise start/end timestamps. Whisper re-transcribes the clipped audio with higher fidelity before the text is handed to LLM.

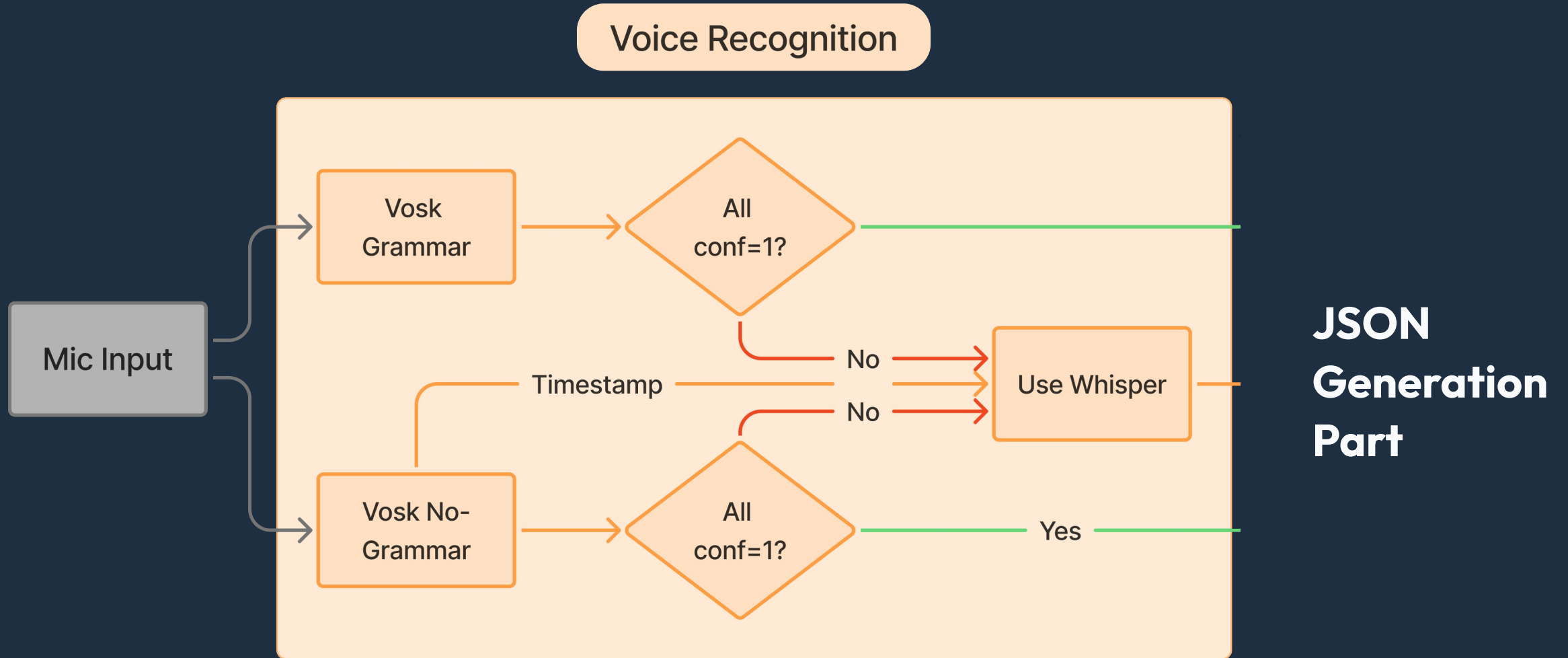
Use When:

Noisy environments, unclear speech, complex or mixed-intent commands.

Flow:

Vosk Timestamps → Whisper → Natural Language

Adaptive Voice Recognition

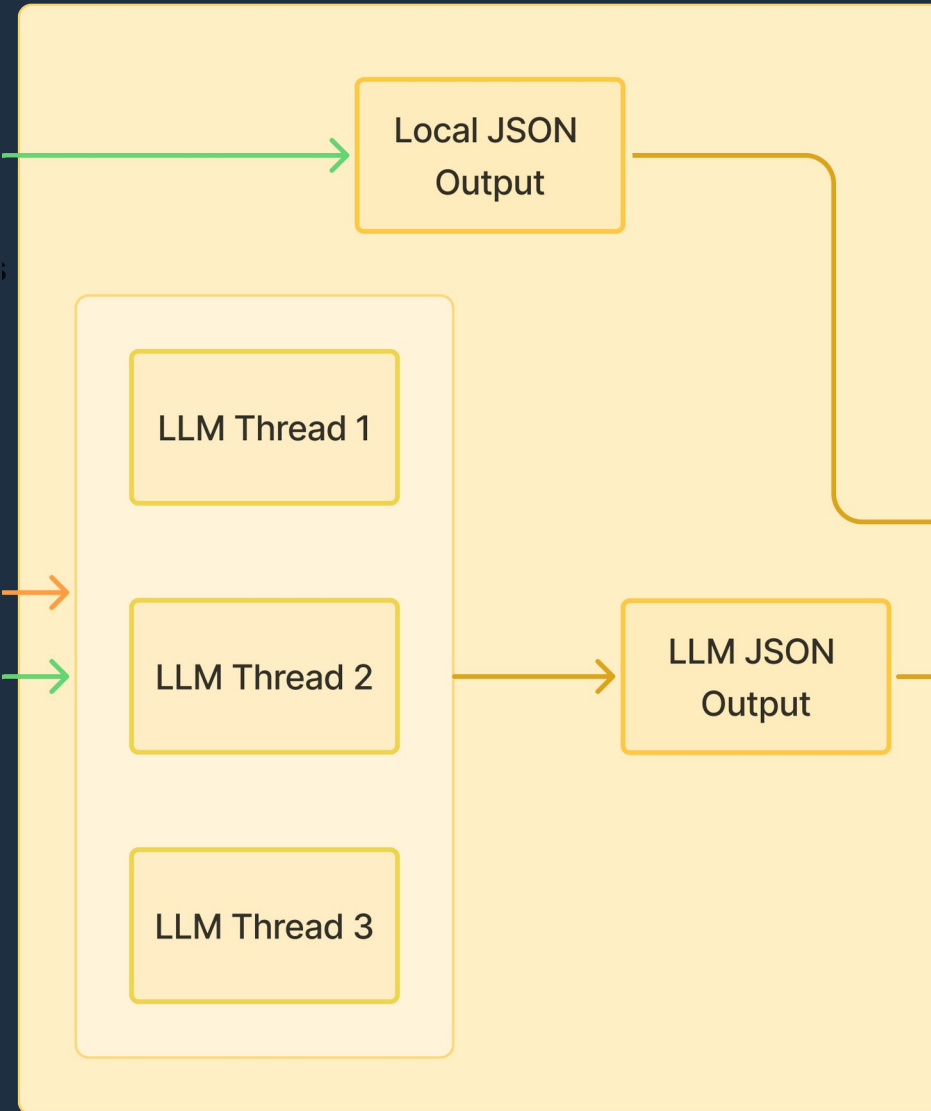


JSON Generation

Fast Path

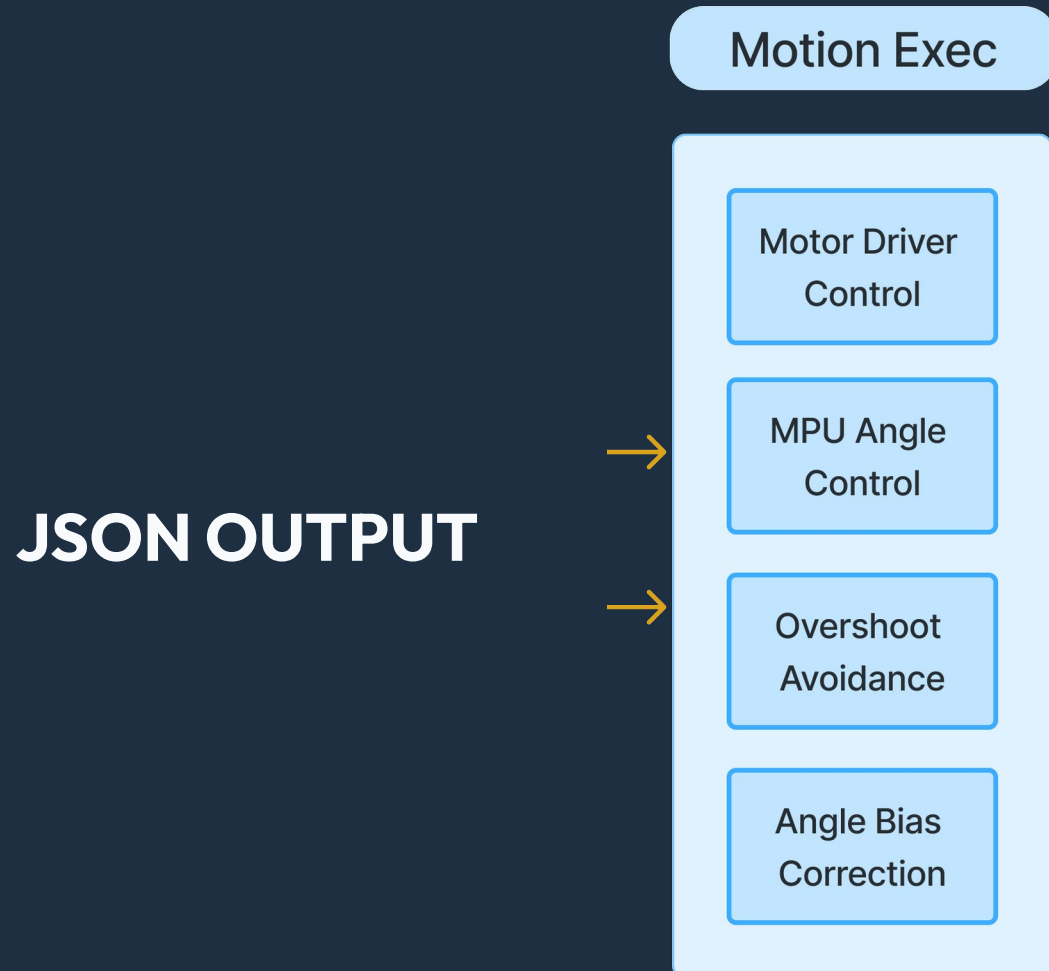
Whisper Path

Direct Path



```
{  
  "action": "...",  
  "duration_ms": ...,  
  "angle_deg": ...,  
  "speed": "...",  
  "id": ...  
}
```

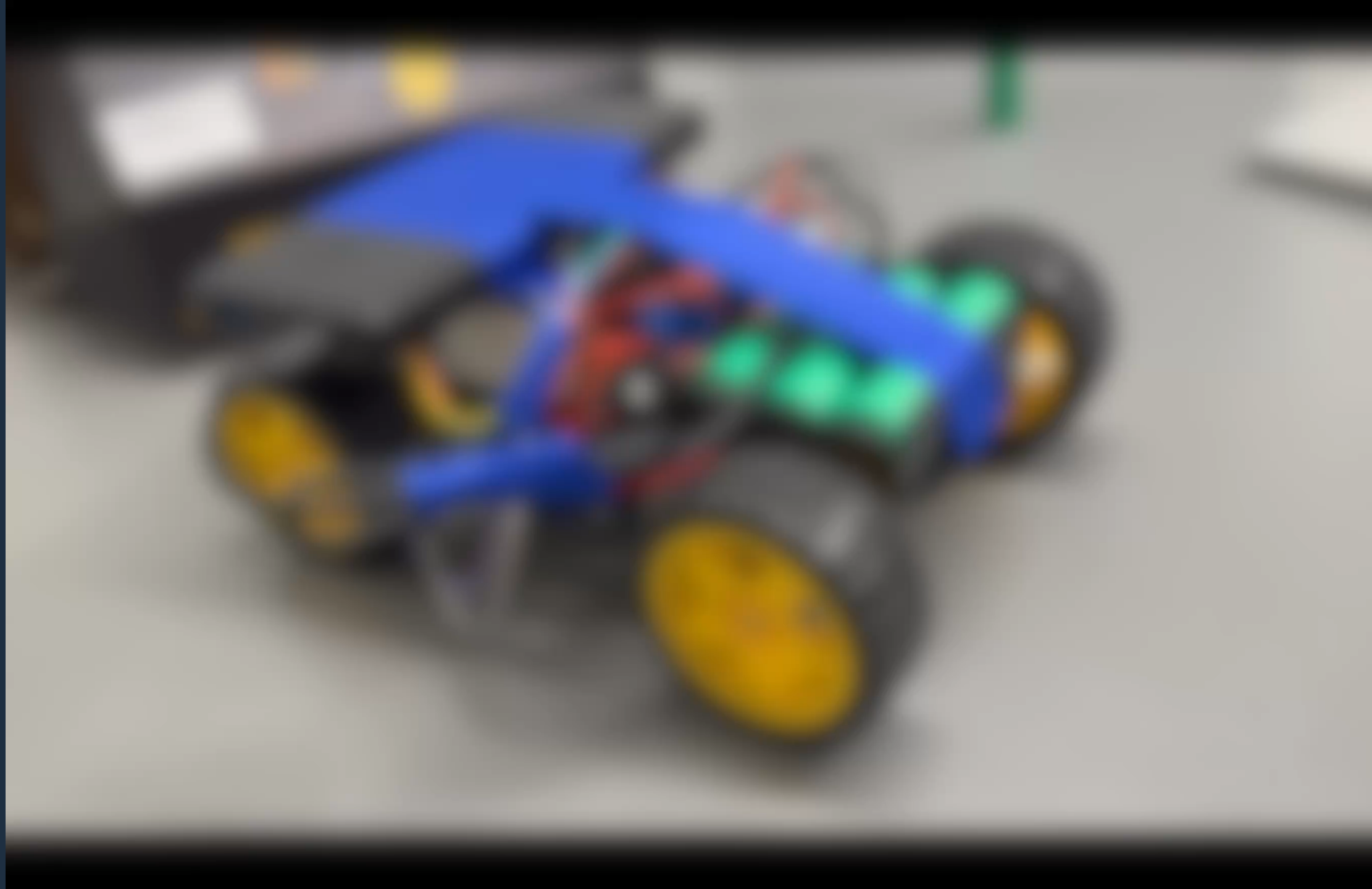
Motion Execution



CAR MOVES!

See in the DEMO -->

Results



Latency

1. Vosk -> JSON: 0.038s

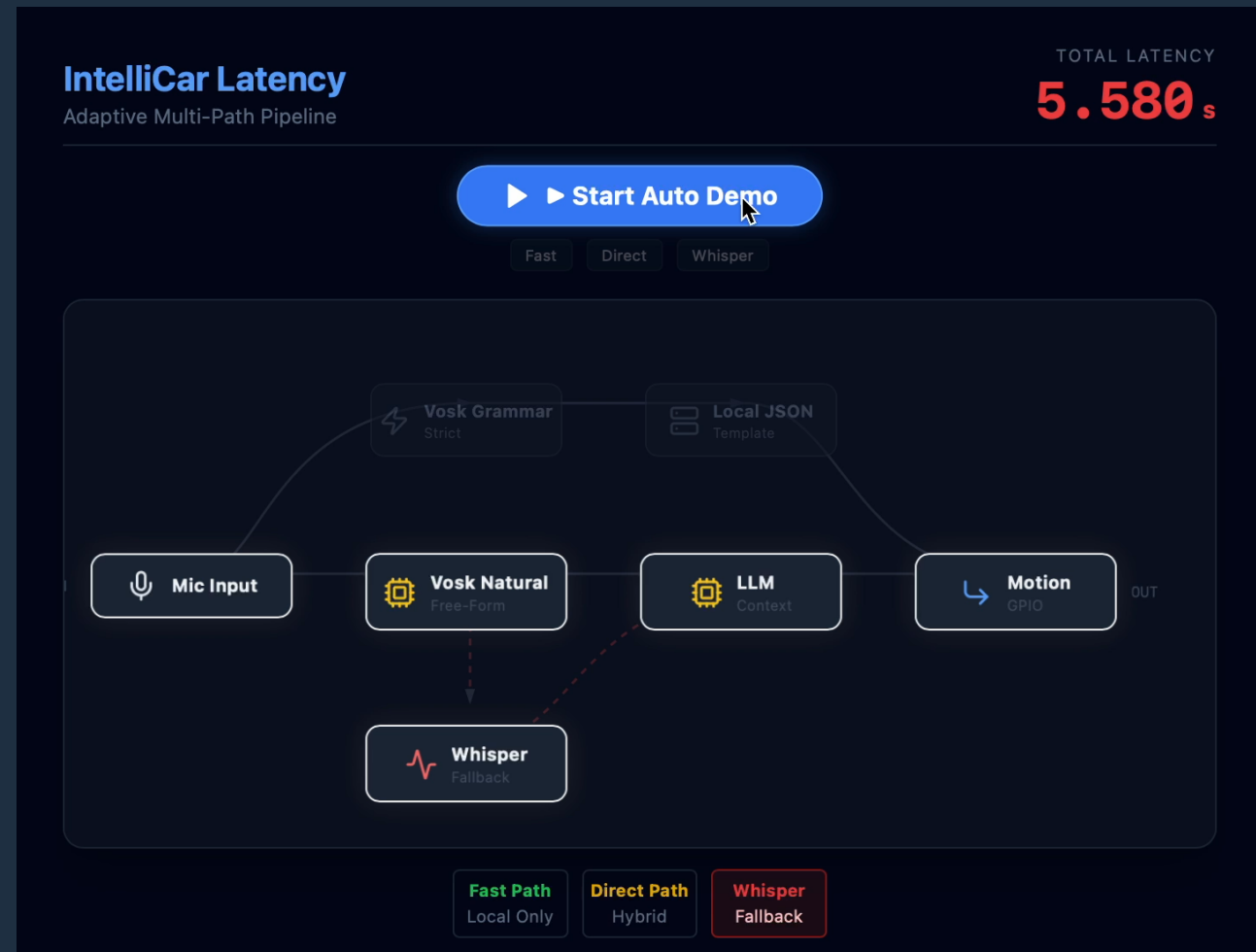
```
✓ Local parser emitted command for segment #1
🕒 Local parse time: 0.60ms
🕒 Segment #1 timing: 1.350s → 2.730s
🕒 Vosk process time: 37.75ms
✓ Local JSON output emitted, segment #1
🕒 Total local processing time: 38.35ms (vosk=
```

2. Vosk -> LLM: 0.650s

```
🕒 Detailed Timing (segment #2):
- Vosk process: 0.062s
- Local Vosk parsing: N/A
- Whisper transcribe: N/A
- OpenAI client init: 0.206s
- LLM call: 0.382s
- Total: 0.650s
```

3. Vosk -> Whisper -> LLM: 5.580s

```
🕒 Detailed Timing (segment #9):
- Vosk process: 0.038s
- Local Vosk parsing: N/A
- Whisper transcribe: 5.007s
- OpenAI client init: 0.205s
- LLM call: 0.330s
- Total: 5.580s
```



Discussion

Challenges & Solutions

Real-time, Stable, Safe

⌚ Making Whisper Fallback Actually Work

We used a ring-buffer + Vosk timestamp trimming so Whisper never waits on long/noisy audio.

→ Eliminates timeouts and keeps fallback responsive.

⌚ Keeping LLM Commands Safe

Strict JSON schema + field-level validation ensures the robot only executes well-formed actions.

→ Protects against hallucinations and malformed outputs.

⌚ Real-Time = Concurrency Management

Bounded queues, a throttled dispatcher, and worker limits keep audio, STT, LLM, and motor control responsive.

→ Prevents overload on the Raspberry Pi.

⌚ Accurate Motion Comes from Sensor Handling

Gyro integration, overshoot guards, and bias correction stabilize multi-step turning.

→ Turns “understanding commands” into reliable execution.

Future Roadmap

Next Step?

Computer Vision

“Follow the red ball”

Context Awareness

Multi-turn conversational context

Power Optimization

Sleep modes and efficient thread management

Application Scenario

Assistive Indoor Mobility Cart

“Take me to the kitchen”

“Go forward slowly”

Factory / Warehouse Voice Navigation Robot

Workers can command hands-free in industrial environments.

Smart Home Voice-Controlled Rover

“Check the living room”

“Come here”

Q & A

Thank you for your attention!

ECE 655L | IntelliCar Team

